

INT315 • CLUSTER COMPUTING

UNIT IV

Running SQL Queries Using Spark SQL

DataFrames, Joins, GroupBy, OrderBy, Aggregates & SQL Wildcards

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Eight syllabus topics, in full depth, with heavy runnable code and seven original diagrams

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit IV

Table of Contents

Unit IV at a Glance	4
1. Importance and Features of Spark SQL	5
1.1 Why Spark SQL Matters	5
1.2 Core Features.....	6
1.3 Importance in the Broader Big Data Ecosystem	7
2. Methods to Convert RDDs to DataFrames	8
2.1 Method 1: Reflection, Using Case Classes	8
2.2 Method 2: Programmatically Specifying a Schema	9
2.3 Choosing Between the Two Methods.....	10
3. Concepts of SparkSQL.....	12
3.1 SparkSession — the Single Unified Entry Point.....	12
3.2 DataFrame, Dataset, and Row	13
3.3 Schema — the Backbone of Everything in This Unit.....	13
3.4 Temporary Views — Bridging DataFrames and SQL Text	14
4. Joins.....	16
4.1 Setting Up Two DataFrames to Join.....	16
4.2 Inner Join — Keep Only the Matches.....	17
4.3 Outer Joins — Keep Unmatched Rows Too	17
4.4 Semi and Anti Joins — Filtering Without Adding Columns	17
4.5 Broadcast Joins — Applying Unit III's Shared Variables.....	18
5. GroupBy.....	20
5.1 Basic GroupBy and Aggregation.....	20
5.2 Grouping by Multiple Columns	21
5.3 Multiple Aggregations in One groupBy.....	21
5.4 Filtering Groups With HAVING	22
6. OrderBy	24
6.1 Basic Ascending and Descending Sorts	24
6.2 Sorting by Multiple Columns.....	24
6.3 What a Global Sort Actually Requires	25
6.4 sortWithinPartitions — a Cheaper Local-Only Alternative	25
7. Aggregate Functions	27
7.1 The Standard SQL Aggregates.....	27
7.2 Collection-Building Aggregates	28

7.3 first() and last() Within a Group	28
7.4 A Preview: Window Functions	28
8. SQL Wildcards	30
8.1 The Two Standard SQL Wildcards	30
8.2 LIKE in the DataFrame API	30
8.3 Case-Insensitive Matching and ILIKE	31
8.4 RLIKE — Full Regular Expression Matching	31
8.5 A Quick Wildcard Reference	32
Unit IV Review	33

Unit IV at a Glance

Unit III gave you the RDD: Spark's foundational, low-level distributed collection, and you learned to transform it using Scala's higher-order functions directly. That power comes at a cost worth naming honestly: an RDD carries no schema, so Spark's Catalyst optimizer (introduced back in Unit I) can't see inside a `map()` or `filter()` closure to optimize it — it just runs your Scala code as written. Spark SQL, and the DataFrame API it's built on, exist to close exactly that gap: give Spark a SCHEMA — named, typed columns — and Catalyst can reason about your query the way a traditional database's query planner reasons about SQL, choosing join strategies, reordering filters, and pruning unread columns automatically, before a single task ever runs.

This unit is the most SQL-flavored one yet: if you already know SQL from any other context, most of what follows will feel immediately familiar, expressed two ways side by side — as DataFrame method calls and as literal SQL text handed to `spark.sql()`. Following Unit III's lead, expect dense, heavy code throughout, plus seven new diagrams covering Spark SQL's architecture, RDD-to-DataFrame conversion, joins, grouping, and sorting.



Handing the Architect a Floor Plan Instead of a Pile of Bricks

An RDD is a truck full of loose bricks -- Spark can move them around exactly as you instruct (`map`, `filter`), but it has no idea what you're BUILDING, so it can't suggest a better construction order. A DataFrame is the same bricks, but now accompanied by an actual floor plan: labeled rooms (columns), known dimensions (types). Hand an architect (Catalyst) that floor plan, and suddenly they can say "actually, build the foundation for the SMALLER wing first, it'll go faster" -- optimizations that were simply invisible when all Spark could see was an unlabeled pile of bricks.

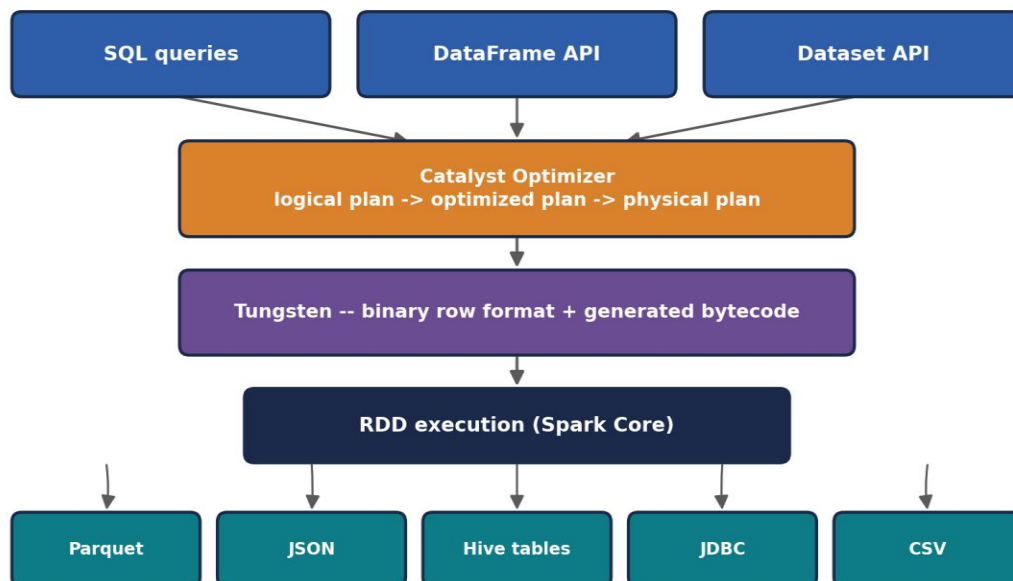
1. Importance and Features of Spark SQL

Spark SQL is the Spark module for working with structured and semi-structured data through SQL and a DataFrame API. This section establishes precisely why it exists and what it brings on top of the RDD foundation from Unit III.

1.1 Why Spark SQL Matters

- **Accessibility** — SQL is the single most widely known data language in the industry. Spark SQL lets analysts, BI tools, and engineers who already know SQL query genuinely massive, distributed datasets without learning Scala's RDD API at all.
- **Automatic optimization** — as previewed in this unit's intro, a schema-aware DataFrame lets Catalyst reorder, prune, and rewrite a query plan the way a traditional RDBMS optimizer does — something Spark simply cannot do with an opaque RDD closure.
- **Unified access to heterogeneous data** — the exact same SQL query or DataFrame code can run against Parquet files, JSON, Hive tables, JDBC-connected relational databases, or CSV, all through one consistent API.
- **A single, well-tested engine, not several** — Spark SQL replaces what used to require separate systems (Hive for SQL-on-Hadoop, a different tool for programmatic access) with one engine serving both use cases identically.

Spark SQL Architecture — From Query to Execution



Every path into Spark SQL -- a literal SQL string, the DataFrame API, or the Dataset API (Scala/Java only) -- converges on the exact same Catalyst Optimizer and Tungsten execution engine from Unit I. This is worth sitting with: writing `spark.sql("SELECT * FROM t WHERE x > 5")` and writing `df.filter(col("x") > 5)` are NOT two different execution paths with different performance characteristics -- they compile down to the identical optimized physical plan.

1.2 Core Features

Feature	What It Means in Practice
Unified data access	One API reads/writes Parquet, JSON, CSV, JDBC, and Hive tables interchangeably
Hive compatibility	Can run existing HiveQL queries and read existing Hive metastore tables directly
Standard connectivity	JDBC/ODBC server mode lets external BI tools (Tableau, Power BI) query Spark directly
Catalyst Optimizer	Cost-based, rule-based query optimization — the same engine covered in Unit I
Tungsten execution	Off-heap binary memory layout and runtime code generation for fast execution
Scalability	The exact same distributed, fault-tolerant execution model from Units I-III underneath

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder()
  .appName("SparkSQLDemo")
  .master("local[*]")
  .getOrCreate()

// Reading a Parquet file and immediately running SQL against it
val df = spark.read.parquet("hdfs:///data/orders.parquet")
df.createOrReplaceTempView("orders")

val highValue = spark.sql("SELECT * FROM orders WHERE amount > 1000")
highValue.show(5)

// The DataFrame API, doing the EXACT same thing, no SQL string at all
val highValueDF = df.filter(df("amount") > 1000)
highValueDF.show(5)
```

1.3 Importance in the Broader Big Data Ecosystem

Recall from earlier units that Hive was built to give analysts SQL access over Hadoop MapReduce, and that Spark SQL's own execution can be dramatically faster on the exact same query, for exactly the reasons Unit I established: in-memory execution plus a real cost-based optimizer, versus Hive's historically MapReduce-based execution. Spark SQL is frequently the FIRST piece of Spark an organization adopts, specifically because it lets existing SQL-literate teams get immediate value without retraining.

Why Spark SQL is often a company's entry point into Spark

Migrating an entire data engineering team from hand-written SQL to Scala RDD code is a genuine organizational cost — new skills, new debugging habits, new hiring criteria. Spark SQL lets a company keep its existing SQL-based reporting and analytics workflow almost entirely intact while gaining Spark's performance and scale underneath, which is precisely why so many real-world Spark adoption stories begin with "we pointed our BI tool at Spark SQL" rather than "we rewrote our pipelines in Scala."

A Universal Translator at the Front Desk

Imagine a huge international company (Spark's execution engine) whose only official internal language is a highly optimized machine code that ordinary employees can't speak (Tungsten bytecode, Catalyst's physical plans). Spark SQL is the polite, extremely competent receptionist at the front desk who accepts requests in ANY visitor's native language -- plain SQL, DataFrame method calls, or Dataset code -- and translates every single one, without exception, into that exact same internal machine language before passing it back to the company floor. No matter which language you walked in speaking, you get identically fast, identically optimized service.

Check yourself — Section 1

Q1. Why does a raw RDD prevent Catalyst from optimizing a query, while a DataFrame does not?

Q2. Name three concrete data sources Spark SQL can read from using the exact same unified API.

Q3. Why do a literal SQL query and an equivalent DataFrame method chain run with identical performance?

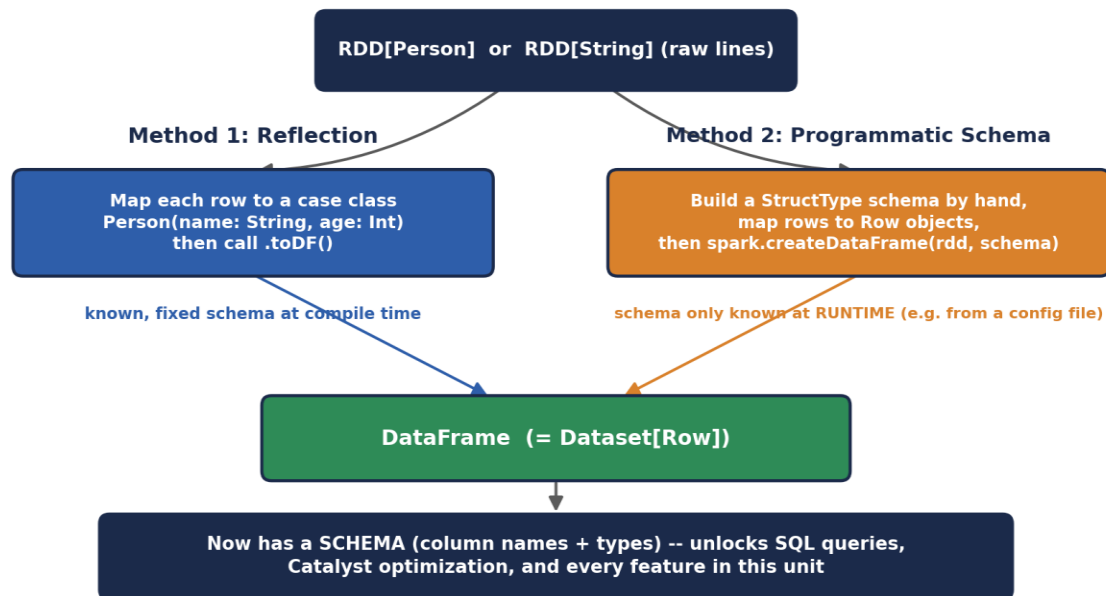
Hands-on Challenge:

Create a small DataFrame (any way you like — see Section 2 for full detail), register it as a temp view, and run the SAME filtering logic two ways: once as a ``spark.sql("SELECT ...")`` string, and once using pure DataFrame method calls. Confirm both produce identical results.

2. Methods to Convert RDDs to DataFrames

Because Unit III's RDD API and this unit's DataFrame API coexist in every real Spark application, moving between them is a routine, everyday operation. Spark provides exactly two ways to do it.

Two Ways to Convert an RDD Into a DataFrame



Two Ways to Convert an RDD Into a DataFrame

2.1 Method 1: Reflection, Using Case Classes

Recall Unit II Section 1.4-1.5: a Scala case class automatically carries its field names and types. Spark SQL uses REFLECTION to read that structure directly and build a matching schema — this is by far the more common approach when your data's shape is known ahead of time.

```
import spark.implicits._ // enables the toDF() conversion used below

case class Employee(name: String, department: String, salary: Double)

val employeeRDD = sc.parallelize(Seq(
  Employee("Aditi", "Engineering", 85000.0),
  Employee("Rahul", "Sales", 72000.0),
  Employee("Meera", "Engineering", 91000.0)
))
```



```

val employeeDF = employeeRDD.toDF()
employeeDF.printSchema()
// root
// |-- name: string (nullable = true)
// |-- department: string (nullable = true)
// |-- salary: double (nullable = true)

employeeDF.show()
// +-----+-----+-----+
// | name| department| salary|
// +-----+-----+-----+
// |Aditi|Engineering|85000.0|
// |Rahul|      Sales|72000.0|
// |Meera|Engineering|91000.0|
// +-----+-----+-----+

// Converting a raw RDD[String] (e.g. from textFile) works the same way,
// once it's been mapped into case class instances
val rawLines = sc.parallelize(Seq("Aditi,Engineering,85000",
    "Rahul,Sales,72000"))
val parsedDF = rawLines.map(_.split(",")).map(arr =>
    Employee(arr(0), arr(1), arr(2).toDouble)
).toDF()

```

2.2 Method 2: Programmatically Specifying a Schema

When the schema isn't known until RUNTIME (perhaps read from a configuration file, or built dynamically based on user input), reflection isn't an option — you build a `StructType` schema by hand and pair it with an RDD of generic `Row` objects.

```

import org.apache.spark.sql.Row
import org.apache.spark.sql.types._

val rawLines = sc.parallelize(Seq("Aditi,Engineering,85000",
    "Rahul,Sales,72000"))

// Build the schema by hand -- StructType is a list of StructField(name, type,
// nullable)
val schema = StructType(Array(
    StructField("name", StringType, nullable = false),
    StructField("department", StringType, nullable = false),
    StructField("salary", DoubleType, nullable = true)
))

// Convert each line into a generic Row matching that schema, in the SAME order
val rowRDD = rawLines.map(_.split(",")).map(arr =>

```

```

    Row(arr(0), arr(1), arr(2).toDouble)
  )

val employeeDF2 = spark.createDataFrame(rowRDD, schema)
employeeDF2.printSchema()
employeeDF2.show()
// identical output to the reflection-based version above

```

⚠ A subtle but common mistake

The columns in each `Row` object must appear in EXACTLY the same order as the fields in the `StructType` schema — Spark matches them positionally, not by name. Swapping the order of two fields in your `Row` construction while forgetting to swap the corresponding `StructField` entries produces a DataFrame that runs without error but silently has the WRONG data under the wrong column names — a genuinely dangerous class of bug precisely because it fails silently.

2.3 Choosing Between the Two Methods

Situation	Preferred Method
Schema known at compile time, fixed structure	Reflection (case class + toDF())
Schema built dynamically from a config file or user input	Programmatic (StructType + createDataFrame)
Working in PySpark (no case classes in Python)	Programmatic (StructType is the standard approach)
Rapid prototyping / exploration in a shell	Reflection — noticeably less code to write

🧐 A Tailored Suit vs. a Made-to-Measure Kit

Reflection (the case class approach) is walking into a tailor who already has your exact measurements on file from a previous visit -- you just say "the usual," and a perfectly fitted schema comes out immediately. The programmatic approach is being handed a made-to-measure kit and a tape measure -- more work, more steps, but it works for a customer (a dataset) whose measurements (schema) genuinely weren't known until they walked in the door.

🏢 Where each approach actually appears in production code

Reflection dominates in application code where the data's shape is a known part of the system design — reading a well-defined event log format, for instance. The programmatic approach shows up constantly in generic, reusable data-loading frameworks — a tool that ingests ARBITRARY CSV files with column definitions supplied by a configuration file has no way to know the schema at compile time, and reaches for StructType by necessity, not preference.

Situation	Preferred Method
 Check yourself — Section 2 Q1. Why can't the reflection-based method be used when a schema is only known at runtime? Q2. What specific danger does the warning above describe, and why does it fail SILENTLY rather than with an error? Q3. In PySpark specifically, why is the programmatic schema approach the standard, rather than a reflection-based one? Hands-on Challenge: Take an RDD of at least 5 raw comma-separated strings representing (product, price, inStock: Boolean) and convert it to a DataFrame BOTH ways: once via a case class and toDF(), and once via a hand-built StructType and createDataFrame(). Confirm both produce identical schemas using printSchema().	

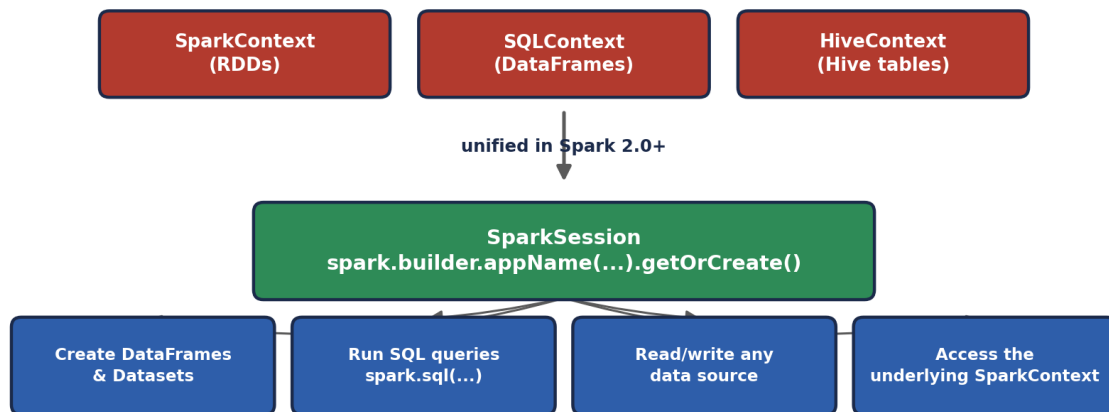
3. Concepts of SparkSQL

This section consolidates the core vocabulary and building blocks Spark SQL is built from -- SparkSession, DataFrame, Dataset, Row, schema, and temporary views -- since every remaining section in this unit uses all of them constantly.

3.1 SparkSession — the Single Unified Entry Point

SparkSession — the Single Unified Entry Point

Before Spark 2.0 -- three separate, overlapping entry points



SparkSession — the Single Unified Entry Point

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder()
  .appName("MySparkSQLApp")
  .master("local[*]")
  .config("spark.sql.shuffle.partitions", "8")
  .enableHiveSupport()           // optional -- adds Hive metastore access
  .getOrCreate()

// The underlying SparkContext (Unit III) is still accessible if needed
val sc = spark.sparkContext

// Recall from Section 5.1 of Unit III: spark-shell builds this FOR you
// automatically, exposed as the pre-built variable 'spark'
```

3.2 DataFrame, Dataset, and Row

RDD vs. DataFrame vs. Dataset

RDD	DataFrame	Dataset
No schema -- just objects	Schema of named, typed COLUMNS	Schema, same as DataFrame
Type-safe at compile time	NOT type-safe -- Row-based, checked at runtime	Type-safe AND Catalyst-optimized
No Catalyst optimization	Full Catalyst + Tungsten optimization	Best of both -- Scala/java only (not Python)
Full control, most verbose	Concise, SQL-like, least verbose	Slightly more verbose than DataFrame

DataFrame == Dataset[Row] -- a DataFrame is simply a Dataset with no compile-time type

RDD vs. DataFrame vs. Dataset

```
// A DataFrame is untyped at compile time -- every row is a generic Row
val df = spark.read.json("hdfs:///data/employees.json")
val firstRow: Row = df.first()
val name: String = firstRow.getAs[String]("name") // manual casting required

// A Dataset[T] is the SAME underlying execution engine, but with a
// compile-time type T -- Scala/Java only, not available in PySpark
case class Employee(name: String, department: String, salary: Double)
val ds: Dataset[Employee] = df.as[Employee]

val firstEmployee: Employee = ds.first()
val name2: String = firstEmployee.name // no casting -- direct, type-checked
field access

// This is the concrete meaning of 'DataFrame = Dataset[Row]'
val backToDF: DataFrame = ds.toDF()
```

3.3 Schema — the Backbone of Everything in This Unit

```
val df = spark.read.option("header", "true").option("inferSchema", "true")
    .csv("hdfs:///data/employees.csv")
```

```
df.printSchema()
df.schema.fields.foreach(f => println(s"${f.name}: ${f.dataType}"))

// Schemas can be explicit instead of inferred -- FASTER (no extra scan of
// the data to guess types) and safer for production pipelines
import org.apache.spark.sql.types._
val explicitSchema = StructType(Array(
  StructField("name", StringType), StructField("department", StringType),
  StructField("salary", DoubleType)
))
val dfExplicit = spark.read.schema(explicitSchema).option("header", "true")
  .csv("hdfs:///data/employees.csv")
```

⚠ inferSchema has a real, often-overlooked cost

Setting `inferSchema=true` forces Spark to perform an EXTRA full (or sampled) pass over the data just to guess column types before the real job even starts — for a large file, this is a genuine, measurable performance cost paid every single time the job runs. Production pipelines reading the same well-understood file format repeatedly should supply an explicit schema instead, both for speed and for protection against a single malformed row silently causing every column in that file to be inferred as a String.

3.4 Temporary Views — Bridging DataFrames and SQL Text

A temp view registers a DataFrame under a name so that literal SQL strings passed to `spark.sql()` can reference it as if it were a table -- this is the exact mechanism Section 1's opening example relied on.

```
val employeeDF = spark.read.json("hdfs:///data/employees.json")

// Session-scoped -- visible only within this SparkSession
employeeDF.createOrReplaceTempView("employees")

// Globally scoped -- visible across MULTIPLE SparkSessions in the same
// application,
// accessed through the special 'global_temp' database
employeeDF.createOrReplaceGlobalTempView("employees_global")

val highEarners = spark.sql("""
  SELECT name, department, salary
  FROM employees
  WHERE salary > 80000
  ORDER BY salary DESC
""")
highEarners.show()
```

```
// See every view/table Spark currently knows about
spark.catalog.listTables().show()
```



A Nickname That Only the SQL Crowd Understands

A `DataFrame` already has a perfectly good name in Scala -- it's just a variable, like `employeeDF`. But the SQL text you hand to ``spark.sql(...)`` doesn't speak Scala variable names; it speaks table names, the way a SQL query always has. ``createOrReplaceTempView`` is giving your `DataFrame` a SPECIAL NICKNAME that the SQL-speaking crowd can recognize and call out to directly, even though, underneath, it's still the exact same `DataFrame` object the whole time -- just introduced under an alias the SQL dialect understands.



Why SparkSession's unification mattered for real teams

Before Spark 2.0's `SparkSession`, real codebases accumulated genuinely confusing boilerplate juggling a `SparkContext` for RDDs, a separate `SQLContext` for `DataFrames`, and a THIRD `HiveContext` for Hive tables — with subtle behavioral differences between them that caused real production bugs. Consolidating all three into one `SparkSession` object was a deliberate simplification that removed an entire category of "which context do I even need here" confusion from every Spark codebase written since.



Check yourself — Section 3

Q1. What's the precise relationship between a `DataFrame` and a `Dataset` — which one is a special case of the other?

Q2. Why does supplying an explicit schema instead of ``inferSchema=true`` matter for production pipeline performance?

Q3. What does ``createOrReplaceTempView`` actually do, and why is it necessary before running a ``spark.sql(...)`` query against a `DataFrame`?

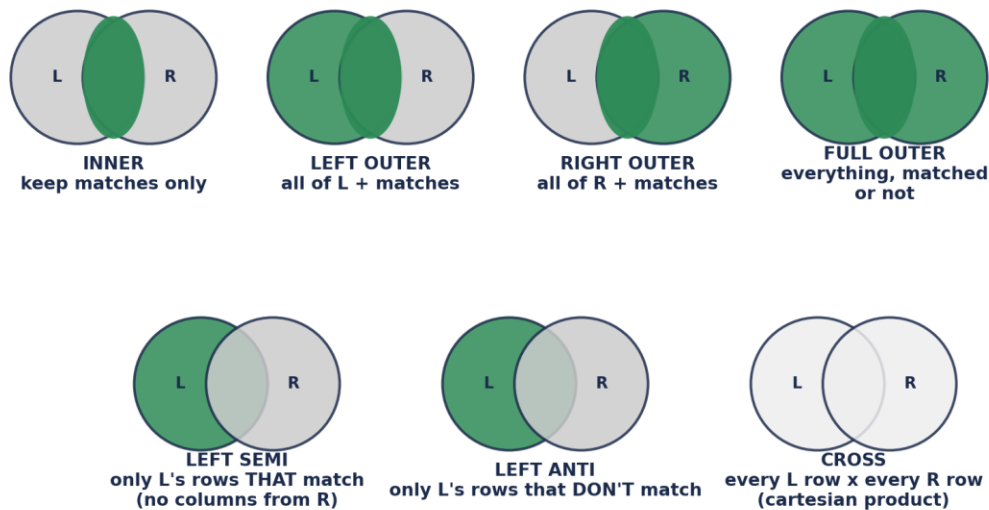
Hands-on Challenge:

Read (or construct) a small `DataFrame`, print its schema, register it as a temp view, and run one query using ``spark.sql()`` and an equivalent query using pure `DataFrame` method calls, confirming they return identical row counts.

4. Joins

A join combines rows from two DataFrames based on a matching condition, exactly as in traditional SQL. Recall Unit III Section 3.4's Pair RDD join -- Spark SQL's join is the schema-aware, Catalyst-optimized evolution of that exact same idea, now expressible in either DataFrame method calls or plain SQL.

Spark SQL / DataFrame Join Types



```
df1.join(df2, df1("id") === df2("id"), "left_outer")
```

Spark SQL / DataFrame Join Types

4.1 Setting Up Two DataFrames to Join

```
import spark.implicits._

val employees = Seq(
  (1, "Aditi", "Engineering"), (2, "Rahul", "Sales"),
  (3, "Meera", "Engineering"), (4, "Zoya", "Marketing")
).toDF("emp_id", "name", "department")

val salaries = Seq(
  (1, 85000), (2, 72000), (3, 91000), (5, 60000) // note: no row for emp_id 4
).toDF("emp_id", "salary") // an extra row for
emp_id 5
```



```
employees.createOrReplaceTempView("employees")
salaries.createOrReplaceTempView("salaries")
```

4.2 Inner Join — Keep Only the Matches

```
// DataFrame API
val innerJoined = employees.join(salaries, "emp_id") // shorthand when the
column name matches
innerJoined.show()
// Zoya (id 4, no salary row) and the id-5 salary (no employee row) both DROP
OUT

// SQL
spark.sql("""
  SELECT e.name, e.department, s.salary
  FROM employees e
  INNER JOIN salaries s ON e.emp_id = s.emp_id
""").show()
```

4.3 Outer Joins — Keep Unmatched Rows Too

```
// LEFT OUTER -- every employee kept, salary is null if missing
employees.join(salaries, Seq("emp_id"), "left_outer").show()
// Zoya appears with salary = null

// RIGHT OUTER -- every salary row kept, employee info is null if missing
employees.join(salaries, Seq("emp_id"), "right_outer").show()
// the id-5 salary row appears with name/department = null

// FULL OUTER -- everything from BOTH sides, matched or not
employees.join(salaries, Seq("emp_id"), "full_outer").show()

// Equivalent SQL
spark.sql("""
  SELECT e.name, e.department, s.salary
  FROM employees e
  LEFT OUTER JOIN salaries s ON e.emp_id = s.emp_id
""").show()
```

4.4 Semi and Anti Joins — Filtering Without Adding Columns

A common need: filter one DataFrame down to only rows that DO (or DON'T) have a match in another DataFrame, WITHOUT actually pulling in any columns from the second DataFrame. Semi and anti joins do exactly this, and are

frequently far more efficient than an inner join followed by a column-drop, because Catalyst can plan them as a pure filter.

```
// LEFT SEMI -- only employees who HAVE a salary record, no salary column added
employees.join(salaries, Seq("emp_id"), "left_semi").show()
// Aditi, Rahul, Meera -- Zoya is excluded, and no 'salary' column appears at
// all

// LEFT ANTI -- only employees who DON'T have a salary record
employees.join(salaries, Seq("emp_id"), "left_anti").show()
// just Zoya

// Equivalent SQL uses EXISTS / NOT EXISTS or IN / NOT IN
spark.sql("""
  SELECT * FROM employees e
  WHERE NOT EXISTS (SELECT 1 FROM salaries s WHERE s.emp_id = e.emp_id)
""").show() // equivalent to left_anti
```

4.5 Broadcast Joins — Applying Unit III's Shared Variables

Recall Unit III Section 6.2's broadcast variables. When one side of a join is SMALL enough to fit comfortably in memory (a lookup table, a dimension table), Spark can broadcast that entire small DataFrame to every executor instead of shuffling both large and small sides across the network — turning an expensive wide-transformation join into a cheap, narrow, per-partition operation.

```
import org.apache.spark.sql.functions.broadcast

// Explicitly hint Spark to broadcast the small 'salaries' DataFrame
val efficient = employees.join(broadcast(salaries), "emp_id")

// Spark's cost-based optimizer will often do this AUTOMATICALLY for small
// tables under spark.sql.autoBroadcastJoinThreshold (default 10MB) --
// the explicit hint above is for when you know better than the optimizer's
// estimate

efficient.explain() // inspect the physical plan -- look for
'BroadcastHashJoin'
// instead of 'SortMergeJoin' to confirm it worked
```

Cross joins are a genuine production hazard

A join with NO condition (or an explicit `.crossJoin()`) produces the full cartesian product of both DataFrames — for two DataFrames of a million rows each, that's a trillion output rows. Spark deliberately requires an explicit `.crossJoin()` call (or a config flag) rather than silently allowing an

accidental cross join through a malformed condition, precisely because this mistake has caused real production outages from runaway resource consumption.



A Wedding Seating Chart

An INNER JOIN is only seating guests who appear on BOTH the bride's list and the groom's list -- anyone on just one list stays home. A LEFT OUTER JOIN seats every one of the bride's guests, and if a matching groom's-side plus-one exists, seats them together -- otherwise that seat next to them stays empty (null). A LEFT SEMI JOIN is simpler still: just tell the caterer a HEADCOUNT of bride's guests who also happen to know someone on the groom's side -- no actual seating assignment, just a filtered count. A CROSS JOIN, seating every single bride's guest next to every single groom's guest in every possible combination, is the seating chart from an actual nightmare -- which is exactly why Spark won't let you generate one by accident.



Why join strategy is a real, common interview topic

"How would you optimize a join between a 500GB fact table and a 2MB dimension table" is a standard Spark interview question, and the expected answer is precisely Section 4.5: broadcast the small side, turning an expensive shuffle join into a cheap broadcast join. Being able to read a `explain()` physical plan and recognize `SortMergeJoin` (expensive, both sides shuffled) versus `BroadcastHashJoin` (cheap, one side broadcast) is considered a baseline competency for a working Spark engineer.



Check yourself — Section 4

Q1. What's the difference between a LEFT OUTER join and a LEFT SEMI join, in terms of which columns end up in the result?

Q2. Why is a broadcast join cheaper than an ordinary shuffle join when one side is small?

Q3. Why does Spark require an explicit method call for a cross join rather than allowing one through a missing join condition silently?

Hands-on Challenge:

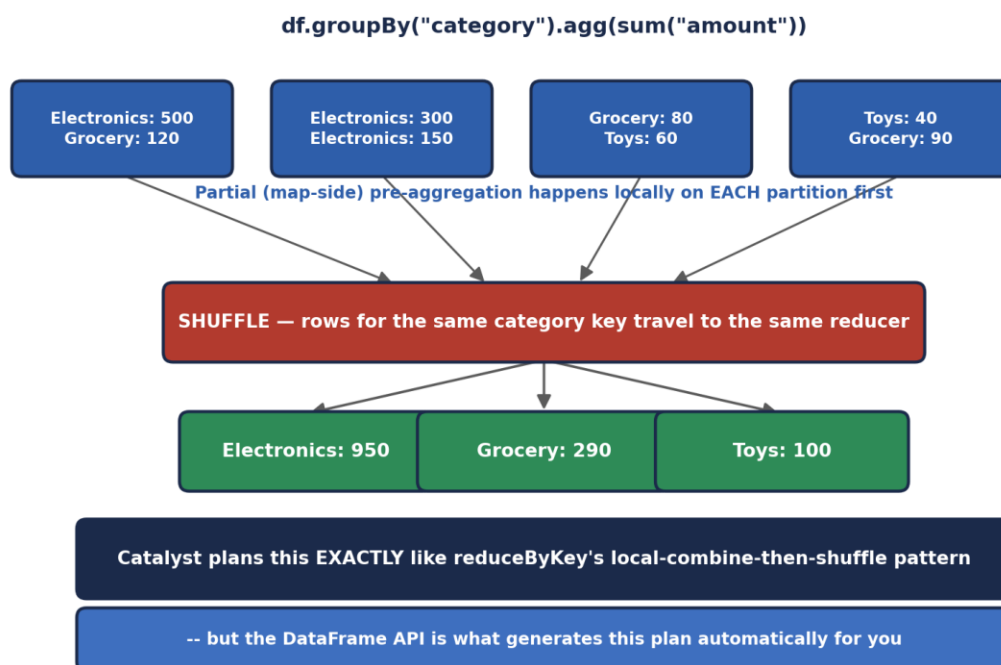
Using the `employees` and `salaries` DataFrames from this section, write and run all of: an inner join, a left outer join, a left semi join, and a left anti join. For each, write down in one sentence how many rows you EXPECT before running it, then confirm.

5. GroupBy

`groupBy` collects rows sharing a common value into groups, so an aggregate function (Section 7) can be computed per group -- exactly the SQL GROUP BY clause, and directly analogous to Unit III's `reduceByKey`, now schema-aware and Catalyst-optimized.

5.1 Basic GroupBy and Aggregation

`groupBy + agg` — Local Pre-Aggregation, Then a Shuffle



groupBy + agg — Local Pre-Aggregation, Then a Shuffle

```
import spark.implicits._
import org.apache.spark.sql.functions._

val sales = Seq(
  ("Electronics", "North", 500), ("Grocery", "North", 120),
  ("Electronics", "South", 300), ("Grocery", "South", 80),
  ("Electronics", "North", 150), ("Toys", "South", 60)
).toDF("category", "region", "amount")

// Single-column grouping
sales.groupBy("category").agg(sum("amount").as("total")).show()
// +-----+-----+
// | category|total|
```

```
// +-----+-----+
// |Electronics| 950|
// |   Grocery| 200|
// |       Toys| 60|
// +-----+-----+

// Equivalent SQL
sales.createOrReplaceTempView("sales")
spark.sql("SELECT category, SUM(amount) AS total FROM sales GROUP BY
category").show()
```

5.2 Grouping by Multiple Columns

```
sales.groupBy("category", "region").agg(sum("amount").as("total")).show()
// +-----+-----+-----+
// | category|region|total|
// +-----+-----+-----+
// |Electronics| North| 650|
// |   Grocery| North| 120|
// |Electronics| South| 300|
// |   Grocery| South| 80|
// |       Toys| South| 60|
// +-----+-----+-----+

spark.sql("""
  SELECT category, region, SUM(amount) AS total
  FROM sales
  GROUP BY category, region
""").show()
```

5.3 Multiple Aggregations in One groupBy

```
sales.groupBy("category").agg(
  sum("amount").as("total"),
  avg("amount").as("average"),
  count("*").as("numTransactions"),
  max("amount").as("biggestSale")
).show()

spark.sql("""
  SELECT category,
    SUM(amount) AS total,
    AVG(amount) AS average,
    COUNT(*) AS numTransactions,
    MAX(amount) AS biggestSale
```

```
FROM sales
GROUP BY category
""").show()
```

5.4 Filtering Groups With HAVING

SQL's WHERE clause filters INDIVIDUAL ROWS before grouping; HAVING filters GROUPS after aggregation -- a distinction that trips up many people learning SQL for the first time, and applies identically here.

```
spark.sql("""
  SELECT category, SUM(amount) AS total
  FROM sales
  GROUP BY category
  HAVING SUM(amount) > 100
""").show()
// Toys (total 60) is excluded -- HAVING filters AFTER aggregation

// DataFrame equivalent -- filter() called AFTER groupBy().agg()
sales.groupBy("category")
  .agg(sum("amount").as("total"))
  .filter($"total" > 100)
  .show()
```

A frequent beginner mix-up

Writing `.filter("amount > 100")` BEFORE `.groupBy()` filters individual transactions before grouping (SQL's WHERE). Writing it AFTER `.groupBy().agg()` filters entire GROUPS based on their aggregated value (SQL's HAVING). These produce genuinely different results, and choosing the wrong one is a common source of subtly incorrect reports.

Sorting Mail Into Bins Before Weighing Them

groupBy is a post office sorting every letter into labeled bins by destination city -- Electronics bin, Grocery bin, Toys bin. *agg()* is then weighing the TOTAL CONTENTS of each bin, one number per bin, not per letter. WHERE is deciding which letters are even allowed into the sorting room in the first place (filtered before any bin exists). HAVING is standing at the end of the day and saying "only keep the bins that ended up weighing more than 100kg" -- a decision that can only be made AFTER the bins are already full and weighed.

Why groupBy + agg is the workhorse of real reporting pipelines

The overwhelming majority of business intelligence dashboards and scheduled reports — daily revenue by region, monthly active users by cohort, error rate by service — boil down to exactly this pattern: group by some dimension, aggregate a metric. Understanding that this compiles down to the same local-pre-aggregation-then-shuffle plan as Unit III's `reduceByKey` is what separates someone who can

WRITE a groupBy query from someone who can also explain why one groupBy query runs in 30 seconds and a superficially similar one takes 20 minutes.

Check yourself — Section 5

Q1. What's the difference between filtering with WHERE and filtering with HAVING, and give an example where they'd produce different results.

Q2. Why does groupBy + agg on a DataFrame follow the same local-pre-aggregation pattern as Unit III's reduceByKey?

Q3. Write, in words, what ``sales.groupBy("category", "region").agg(count("*"))`` computes.

Hands-on Challenge:

Using the ``sales`` DataFrame from this section, write a query that groups by BOTH category and region, computes the total AND average amount per group, and keeps only groups with a total above 100 — do this once with the DataFrame API and once with SQL, confirming both produce the same rows.

6. OrderBy

`orderBy` (and its alias `sort`) produces a GLOBALLY sorted result across the entire `DataFrame` -- not just sorted within each partition. This section covers the syntax and, importantly, what actually has to happen underneath for a distributed sort to be globally correct.

6.1 Basic Ascending and Descending Sorts

```
import spark.implicits._
import org.apache.spark.sql.functions._

val products = Seq(
  ("Laptop", 75000), ("Mouse", 500), ("Keyboard", 1500),
  ("Monitor", 12000), ("Webcam", 2500)
).toDF("product", "price")

// Ascending (the default)
products.orderBy("price").show()

// Descending
products.orderBy(desc("price")).show()
products.orderBy($"price".desc).show() // equivalent, alternate syntax

// SQL
products.createOrReplaceTempView("products")
spark.sql("SELECT * FROM products ORDER BY price DESC").show()
```

6.2 Sorting by Multiple Columns

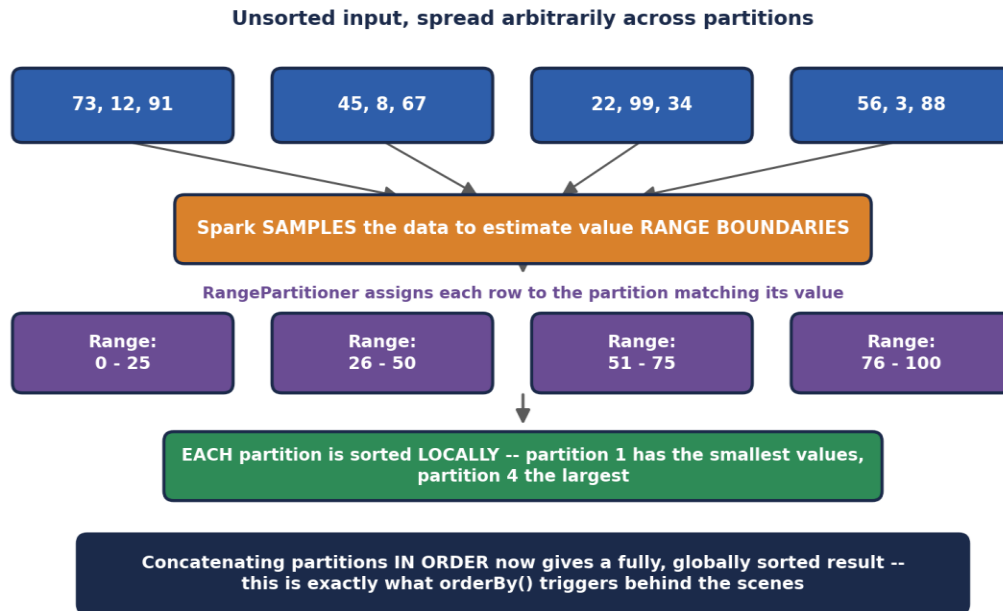
```
val employees = Seq(
  ("Aditi", "Engineering", 85000), ("Rahul", "Sales", 72000),
  ("Meera", "Engineering", 91000), ("Zoya", "Sales", 72000)
).toDF("name", "department", "salary")

// Sort by department ascending, THEN by salary descending WITHIN each
// department
employees.orderBy(asc("department"), desc("salary")).show()

spark.sql("""
  SELECT * FROM employees
  ORDER BY department ASC, salary DESC
""").show()
```


6.3 What a Global Sort Actually Requires

orderBy() — a Global Sort via Range Partitioning



orderBy() — a Global Sort via Range Partitioning

This is worth understanding precisely because it's a genuinely different (and more expensive) pattern than groupBy's shuffle from Section 5. A GLOBAL sort requires knowing, for every row, which OTHER partition's rows come before or after it — which isn't knowable just by looking at one partition in isolation. Spark solves this with RANGE PARTITIONING: sample the data to estimate value boundaries, reassign every row to a partition based on which range its value falls into, sort each partition locally, and the partitions concatenated in order are now globally sorted.

```
// You can inspect this yourself
val sorted = products.orderBy("price")
println(sorted.rdd.getNumPartitions)

sorted.explain()
// Look for 'Exchange rangepartitioning' in the physical plan --
// that's this exact range-partitioning shuffle happening
```

6.4 sortWithinPartitions — a Cheaper Local-Only Alternative

When a genuinely GLOBAL order isn't required -- only that each partition's own output happens to be locally sorted, useful before certain downstream writes -- sortWithinPartitions avoids the expensive shuffle entirely.

```
// Cheap -- NO shuffle, each partition sorted independently, in place
val locallySorted = products.sortWithinPartitions("price")

// Expensive -- REQUIRES a full shuffle for a true global order
val globallySorted = products.orderBy("price")
```

A performance trap worth remembering

Calling `.orderBy()` when only a locally-sorted or approximately-sorted output is actually needed is a common, avoidable source of unnecessary shuffle cost — always ask whether the downstream consumer of the sorted data genuinely needs a strict GLOBAL order (a report meant to be read top-to-bottom) or would be equally well served by `.sortWithinPartitions()` (data being written out for a system that will re-sort or doesn't care about cross-partition order).

Sorting an Entire Library vs. Sorting Each Shelf

sortWithinPartitions is asking each of ten librarians to alphabetize ONLY the shelf directly in front of them -- fast, parallel, no communication between librarians required. orderBy (a true global sort) is asking for the ENTIRE library to be alphabetized as one continuous sequence from the front door to the back wall -- which means librarians first have to huddle and agree on which books physically belong on shelf 1 versus shelf 10 (the range-partitioning sample step) BEFORE any actual shelving can happen, and only then does each librarian sort their now-correctly-assigned shelf.

Why this distinction matters for real pipeline costs

A full global `.orderBy()` on a genuinely large dataset is one of the more expensive operations available in Spark SQL, precisely because of the sampling-plus-shuffle overhead described above — production pipelines writing large output files commonly use `.sortWithinPartitions()` (or skip sorting entirely) unless a strict global order is a real, stated requirement, specifically to avoid paying for a guarantee nobody downstream actually needs.

Check yourself — Section 6

Q1. Why can't a global sort be computed by simply having each partition sort its own data independently?

Q2. What specific step does Spark perform BEFORE assigning rows to their final sorted partition, and why is that step necessary?

Q3. When would `.sortWithinPartitions()` be an appropriate, cheaper alternative to `.orderBy()`?

Hands-on Challenge:

Using the `employees` DataFrame from this section, write a query sorting by department ascending and salary descending, once with the DataFrame API and once with SQL. Then call `.explain()` on the DataFrame version and find the line indicating a range-partitioning exchange.

7. Aggregate Functions

Section 5 used a handful of aggregate functions in passing -- this section catalogs the full set available in Spark SQL, from the familiar SQL basics through the more specialized collection-building and statistical functions.

7.1 The Standard SQL Aggregates

Function	Signature	What It Computes
count	count(col) / count("*")	Number of non-null values in a column, or total rows with "*"
sum	sum(col)	Sum of all values
avg / mean	avg(col)	Arithmetic mean
min / max	min(col) / max(col)	Smallest / largest value
countDistinct	countDistinct(col)	Number of DISTINCT non-null values
stddev / variance	stddev(col) / variance(col)	Standard deviation / variance

```
import org.apache.spark.sql.functions._

val sales = Seq(
  ("Electronics", 500), ("Grocery", 120), ("Electronics", 500),
  ("Electronics", 300), ("Grocery", 80)
).toDF("category", "amount")

sales.select(
  count("*").as("totalRows"),
  sum("amount").as("totalAmount"),
  avg("amount").as("averageAmount"),
  min("amount").as("minAmount"),
  max("amount").as("maxAmount"),
  countDistinct("category").as("distinctCategories"),
  round(stddev("amount"), 2).as("stdDev")
).show()

// SQL equivalent
sales.createOrReplaceTempView("sales")
spark.sql("""
  SELECT COUNT(*) AS totalRows, SUM(amount) AS totalAmount,
         AVG(amount) AS averageAmount, COUNT(DISTINCT category) AS
distinctCategories
  FROM sales
""").show()
```

7.2 Collection-Building Aggregates

Instead of reducing a group down to a single number, these functions collect a group's values into an actual list or set -- extremely useful for building denormalized, reporting-friendly structures.

```
sales.groupBy("category").agg(
  collect_list("amount").as("allAmounts"), // keeps duplicates, order not
  collect_set("amount").as("uniqueAmounts") // deduplicated
).show(false)
// +-----+-----+-----+
// | category| allAmounts| uniqueAmounts|
// +-----+-----+-----+
// |Electronics|[500, 500, 300]| [500, 300]|
// | Grocery| [120, 80]| [120, 80]|
// +-----+-----+-----+
```

7.3 first() and last() Within a Group

```
val events = Seq(
  ("user1", "login", 1), ("user1", "click", 2), ("user1", "logout", 3),
  ("user2", "login", 1), ("user2", "logout", 2)
).toDF("user", "action", "sequenceNum")

// first()/last() without an explicit order are NOT deterministic on their own -
// always combine with orderBy or a window function for a reliable result
val ordered = events.orderBy("user", "sequenceNum")
ordered.groupBy("user").agg(
  first("action").as("firstAction"),
  last("action").as("lastAction")
).show()
```

first()/last() without ordering is a real correctness trap

Because a DataFrame's rows have no inherent order across a distributed cluster, calling `first()` or `last()` inside a `groupBy().agg()` WITHOUT first guaranteeing a deterministic order (via `orderBy` beforehand, or a proper window function) can return a DIFFERENT row on different runs of the exact same query — a subtle, hard-to-catch bug that has caused real production incidents.

7.4 A Preview: Window Functions

Standard aggregates collapse a group into ONE row. Sometimes you need a per-group aggregate value attached to EVERY original row instead — a running total, a rank within a group — without collapsing anything. This is what window functions provide, briefly previewed here as a bridge to later, deeper coverage.

```
import org.apache.spark.sql.expressions.Window

val employees = Seq(
  ("Aditi", "Engineering", 85000), ("Rahul", "Sales", 72000),
  ("Meera", "Engineering", 91000), ("Zoya", "Sales", 88000)
).toDF("name", "department", "salary")

val byDept = Window.partitionBy("department").orderBy(desc("salary"))

employees.withColumn("rankInDept", rank().over(byDept)).show()
// every ORIGINAL row is kept, each now carrying its own rank -- unlike
// groupBy().agg(), which would have collapsed each department into one row
```



A Class Average vs. Everyone's Individual Report Card

sum() and avg() inside a groupBy produce a CLASS AVERAGE -- one number, representing the whole group, and the individual students disappear into that single summary figure. A window function is handing back EVERY individual student's own report card, but with the class average (or their rank in class) PRINTED ON IT as extra context -- nobody disappears, but everyone now also carries a bit of group-level information alongside their own personal row.



Why collect_list/collect_set show up in real ETL work

A common real-world pattern: flattening a normalized `orders` and `order\_items` pair of tables into one denormalized row per order, with `collect\_list` gathering all of that order's line items into a single array column — a shape that's often far more convenient for a downstream API or a document-oriented database than the original relational structure. Recognizing when a job actually needs collection-building aggregates instead of scalar ones is a practical, everyday Spark SQL skill.



Check yourself — Section 7

Q1. What's the difference between `collect\_list` and `collect\_set`?

Q2. Why is calling `first()` inside a `groupBy().agg()` risky without a preceding `orderBy`?

Q3. What's the key difference between a normal aggregate function and a window function, in terms of how many rows come out?

Hands-on Challenge:

Using the `sales` DataFrame from this section, compute count, sum, average, and countDistinct in one query, then separately use `collect\_list` to gather all amounts per category, and finally add a window function ranking each individual sale by amount within its category (without collapsing any rows).

8. SQL Wildcards

This closing section covers PATTERN MATCHING on string columns -- the LIKE operator and its wildcard characters, plus Spark SQL's extension into full regular expressions, both usable identically in SQL text and in the DataFrame API.

8.1 The Two Standard SQL Wildcards

Wildcard	Meaning
% (percent)	Matches ZERO or more of any character
_ (underscore)	Matches EXACTLY ONE of any character

```
val products = Seq(
  "Wireless Mouse", "Wireless Keyboard", "USB Mouse",
  "Bluetooth Speaker", "Mouse Pad", "Wired Mouse"
).toDF("product")
products.createOrReplaceTempView("products")

// % -- starts with "Wireless"
spark.sql("SELECT * FROM products WHERE product LIKE 'Wireless%').show()
// Wireless Mouse, Wireless Keyboard

// % -- contains "Mouse" anywhere
spark.sql("SELECT * FROM products WHERE product LIKE '%Mouse%').show()
// Wireless Mouse, USB Mouse, Mouse Pad, Wired Mouse

// % -- ends with "Mouse"
spark.sql("SELECT * FROM products WHERE product LIKE '%Mouse']").show()
// Wireless Mouse, USB Mouse, Wired Mouse (NOT 'Mouse Pad')

// _ -- exactly one character in that position
spark.sql("SELECT * FROM products WHERE product LIKE '_SB Mouse').show()
// USB Mouse (the _ matches exactly the 'U')
```

8.2 LIKE in the DataFrame API

```
import org.apache.spark.sql.functions._

products.filter($"product".like("Wireless%")).show()
products.filter(col("product").like("%Mouse%")).show()

// NOT LIKE
```

```
spark.sql("SELECT * FROM products WHERE product NOT LIKE '%Mouse%'").show()
products.filter(! $"product".like("%Mouse%")).show()
```

8.3 Case-Insensitive Matching and ILIKE

```
// LIKE is case-SENSITIVE by default
spark.sql("SELECT * FROM products WHERE product LIKE '%mouse%'").show()
// returns NOTHING -- lowercase 'mouse' doesn't match 'Mouse'

// ILIKE -- case-INSENSITIVE LIKE (Spark 3.3+)
spark.sql("SELECT * FROM products WHERE product ILIKE '%mouse%'").show()
// now correctly matches all four "Mouse" products

// The traditional, portable way to get the same effect
spark.sql("SELECT * FROM products WHERE LOWER(product) LIKE '%mouse%'").show()
```

8.4 RLIKE — Full Regular Expression Matching

For pattern-matching needs beyond what % and _ can express, Spark SQL supports RLIKE, backed by full regular expression syntax -- directly useful for validating formats (emails, phone numbers) or matching more complex patterns than simple substring wildcards allow.

```
val contacts = Seq(
  "aditi@company.com", "not-an-email", "rahul.k@company.co.in", "12345"
).toDF("contact")
contacts.createOrReplaceTempView("contacts")

// RLIKE -- basic email-shape validation
spark.sql("""
  SELECT * FROM contacts
  WHERE contact RLIKE '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
""").show()
// aditi@company.com, rahul.k@company.co.in

// DataFrame API equivalent
contacts.filter($"contact".rlike("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")).show()

// Matching product codes like exactly 3 letters followed by 4 digits
val codes = Seq("ABC1234", "AB12345", "XYZ0001").toDF("code")
codes.filter($"code".rlike("^[A-Z]{3}[0-9]{4}$")).show()
// ABC1234, XYZ0001
```

8.5 A Quick Wildcard Reference

Pattern	Matches
'A%'	Starts with A
'%A'	Ends with A
'%A%'	Contains A anywhere
'A_C'	Starts with A, ends with C, exactly one character in between
'A__%'	Starts with A, at least 3 characters total
'[Rr]like'	RLIKE regex — case-sensitive alternation via a character class



A Detective's Two Different Magnifying Glasses

LIKE with % and _ is a detective's basic magnifying glass -- perfectly good for "find anything starting with A" or "find a 5-letter word where the 3rd letter is Q," simple shape-matching that most everyday searches need. RLIKE is swapping that basic glass for a full forensic toolkit -- capable of expressing genuinely intricate patterns ("a valid email shape," "three letters then four digits, nothing else") that no amount of clever %-and-_-combining could ever express. Reach for the simple magnifying glass first; only pull out the forensic toolkit when the pattern genuinely needs it.



Where wildcard matching shows up in real data work

Data quality and validation pipelines lean on RLIKE constantly — flagging malformed email addresses, phone numbers, or product codes before they propagate downstream. Simple LIKE patterns are the everyday tool for category-style filtering in ad hoc analysis — "find every product whose name suggests it's wireless" — precisely the kind of quick, exploratory query a data analyst runs dozens of times a day directly from a SQL-connected BI tool sitting on top of Spark SQL.



Check yourself — Section 8

- Q1. What's the difference between the % and _ wildcards in a LIKE pattern?
- Q2. Why does `LIKE '%mouse%'` fail to match "Mouse Pad", and what are two ways to fix that?
- Q3. When would you reach for RLIKE instead of LIKE, and give one concrete example pattern that LIKE alone couldn't express?

Hands-on Challenge:

Using the `products` DataFrame from this section, write four separate LIKE queries — starts-with, ends-with, contains, and exactly-one-wildcard-character — then write one RLIKE query validating that a product name contains only letters and spaces (no digits or symbols).

Unit IV Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. Why can Catalyst optimize a DataFrame query but not an equivalent chain of raw RDD transformations?
2. Q2. What's the difference between a LEFT OUTER join and a LEFT SEMI join, and when would you prefer the semi join?
3. Q3. Why does a global orderBy() require a more expensive shuffle than a groupBy().agg(), conceptually?

Answer key (cover before peeking)

A1: A DataFrame carries a SCHEMA (named, typed columns) that Catalyst can inspect and reason about — reordering filters, choosing join strategies, pruning columns. A raw RDD transformation is an opaque Scala closure; Spark has no visibility into what it does, so there's nothing for Catalyst to optimize.

A2: A LEFT OUTER join keeps every row from the left side and adds matching columns from the right (null if no match). A LEFT SEMI join keeps only left-side rows that HAVE a match, but adds NO columns from the right at all. Prefer semi join when you only need to filter, not enrich — it's typically cheaper.

A3: groupBy().agg() only needs rows sharing the SAME key to end up together, which a hash-based shuffle achieves directly. A global orderBy() needs every row correctly positioned relative to EVERY other row across the whole dataset, requiring Spark to first sample the data to estimate range boundaries before it can even begin the shuffle — an extra step groupBy doesn't need.

Hands-On Exercise

Complete, end to end, using spark-shell or a notebook with a SparkSession available:

4. Create two small DataFrames representing customers (customer_id, name, city) and orders (order_id, customer_id, product, amount) — using EITHER conversion method from Section 2 for at least one of them.
5. Register both as temp views and write one query using spark.sql() that joins them (Section 4) and computes total spend per customer using groupBy + agg (Section 5).
6. Sort the result by total spend descending (Section 6), and use LIMIT / take() to show only the top 3 customers.
7. Add a query using collect_list (Section 7) to show, per customer, the full list of products they've ordered.
8. Finally, use a LIKE or RLIKE wildcard query (Section 8) to find all customers whose city name starts with a specific letter, or all products matching a naming pattern of your choosing.

 What this exercise is really testing

This single exercise deliberately exercises every syllabus topic in this unit at once: converting data into DataFrames (Section 2), running real SQL and DataFrame queries side by side (Sections 1 and 3), joining two datasets (Section 4), grouping and aggregating (Sections 5 and 7), sorting (Section 6), and pattern matching (Section 8) — a complete, realistic mini-analytics pipeline built entirely from this unit's material.